Оглавление

Введение.....2

1. Символьные вычисления. Библиотека SymPy 3

2. Работа с матрицами и массивами. Библиотека NumPy 7

3. Вейвлеты. Библиотека PyWavelets 13

4. Дискретное вейвлет-преобразование..... 16

6. Вейвлет-пакеты..... 31

7. Вейвлет-пакеты и изображения 34

Введение

Вейвлеты — это удобный инструмент для анализа и преобразования данных. Они позволяют работать с данными большого объема, такими как тексты, изображения, аудио- и видеофайлы. Вейвлеты представляют собой набор математических функций, которые применяются к данным для извлечения информации о них.

Одним из основных преимуществ вейвлетов является возможность обработки данных без необходимости их полного понимания. Это особенно полезно при работе с большими объемами данных, когда невозможно провести детальный анализ каждого элемента.

Кроме того, вейвлеты позволяют проводить анализ данных в реальном времени, что делает их незаменимыми инструментами для многих задач. Например, они используются в системах безопасности для распознавания лиц или определения настроения людей на фотографиях.

Также стоит отметить, что вейвлеты позволяют проводить анализ данных с высокой точностью. Это связано с тем, что они основаны на математических алгоритмах, которые учитывают множество факторов и особенностей данных.

В целом, использование вейвлетов позволяет существенно ускорить процесс анализа данных и повысить точность результатов. Они являются незаменимыми инструментами для работы с большими объемами данных и находят широкое применение в различных областях, таких как финансы, медицина, наука и технологии.

1. Символьные вычисления. Библиотека SymPy

Для выполнения символьных вычислений в Python используется библиотека SymPy. В ней реализованы операции, которые позволяют решать различные задачи:

- упрощение математических выражений;
- вычисление пределов;
- вычисление производных;
- интегрирование;
- разложение функции в ряд;
- нахождение сумм ряда;
- операции с матрицами;
- построение графиков функции

и другие.

Для использования библиотеки SymPy требуется сначала её подключить:

```
from sympy import*
```

Для работы с символьными переменными их необходимо сначала задать с помощью команд Symbol() для одной переменной, symbols() или var() для нескольких

```
a1 = Symbol('a1')
var('b1 c1')
```

В качестве примера рассмотрим сложение дробей для символьных и обычных переменных

```
a1 = Rational(1,2)
print(a1)
b1 = Rational(1,3)
print(b1)
c1 = a1+b1
print(c1)
```

1/2
1/3
5/6

Заметим, что в функции `Rational()` имеются 2 целочисленных аргумента — числитель и знаменатель дроби. Сравните полученные результаты с теми, что получаются при выполнении обычных арифметических операций в Python

```
a2 = 1/2
print(a2)
b2 = 1/3
print(b2)
c2 = a2+b2
print(c2)
```

0.5

0.3333333333333333
0.8333333333333333

В Python имеется возможность работы с бесконечностью (обозначаемой `oo`):

```
print(1/oo)
```

0

Вычисление пределов производится с помощью функции `limit(функция, переменная, точка)`. Значение параметра `точка` может быть и числом, и бесконечностью.

```
x = Symbol('x')
print(limit(sin(x)/x, x, 0)
print(limit(sin(x)/x, x, oo)
```

1
0

Продифференцировать любое выражение можно, используя функцию `diff(функция, переменная, порядок)`. Параметр `порядок` определяет порядок производной и не является обязательным (по умолчанию равен 1).

```
y = Symbol('y')
print(diff(sin(x)**2, x))
print(diff(sin(x)**2, x, 1))
print(diff(sin(x)**2, x, 2))
print(diff((sin(x)+cos(x*y))**2,x))
print(diff((sin(x)+cos(x*y))**2,y))
```

2*sin(x)*cos(x)
2*sin(x)*cos(x)
2*(-sin(x)**2 + cos(x)**2)
(-2*y*sin(x*y) + 2*cos(x))*(sin(x) + cos(x*y))
-2*x*(sin(x) + cos(x*y))*sin(x*y)

В SymPy можно проводить вычисление как неопределённых интегралов, применяя функцию `integrate(функция, переменная)`, так и определённых `integrate(функция,(переменная, a,b))`, где добавлены пределы интегрирования.

```
print(integrate(sin(x)**2, x))
print(integrate(sin(x)**2, (x, 0, pi/2)))
print(integrate(sin(x)**2, (x, 0, pi/2)).evalf())
print(integrate(1/x**3, (x, 1,oo)))
```

x/2 - sin(x)*cos(x)/2
pi/4
0.785398163397448
1/2

Для разложения функции в степенной ряд можно использовать функцию `series(функция, переменная, точка, порядок)`.

```
f = exp(2*x)+ln(1+x/2)
print(series(f,x,0,4))
```

1 + 5*x/2 + 15*x**2/8 + 11*x**3/8 + O(x**4)

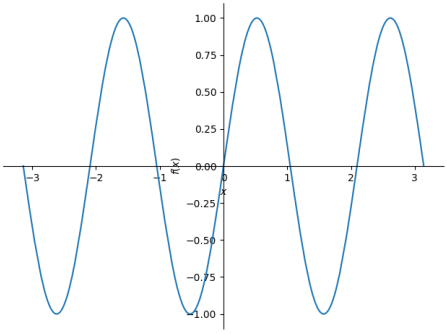
Для разложения функции в ряд Фурье применяется функция `fourier_series(функция, (переменная, a, b))`.

```
f = x**2+2*x
s = fourier_series(f,(x,-2,2))
print(s.truncate(4))
```

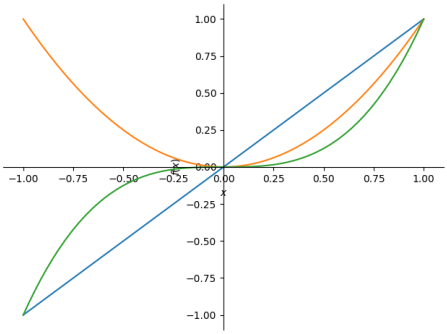
8*sin(pi*x/2)/pi - 4*sin(pi*x)/pi + 8*sin(3*pi*x/2)/(3*pi) - 16*cos(pi*x/2)/pi**2 + 4*cos(pi*x)/pi**2 - 16*cos(3*pi*x/2)/(9*pi**2) + 4/3

В SymPy имеется возможность построения двумерных и трёхмерных графиков при помощи функций `plot()` и `plot3d()`.

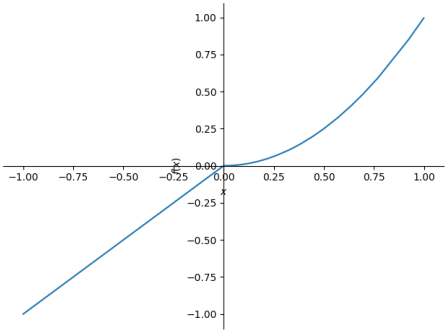
```
plot(sin(3*x),(x,-pi,pi))
```




```
plot(x,x**2,x**3,(x,-1,1))
```




```
f=Piecewise((x,(x<=0)&(x>-1)),(x**2,(x<=1)&(x>0)))
plot(f)
```



Задания.

1. Для функции $f(t) = t^2 + 2sin(\pi \cdot t) + 4cos(3\pi \cdot t)$
   a. постройте график функции на отрезке [-4; 4];
   b. найдите производную функции *f'*(*t*);
   c. вычислите производную в точке *t* = 2;
   d. найдите неопределённый интеграл $\int f\,(t)dt$
   e. вычислите определённый интеграл $\int_{-4}^{4} f\,(t)dt$ непосредственно и численно.
2. Степенной ряд
   a. Разложите функцию $f(t) = t^2 + 2sin(\pi \cdot t) + 4cos(3\pi \cdot t)$в степенной ряд *s*(*t*) в точке x = 0, используя 3 члена разложения;
   b. Постройте на одном рисунке графики *f*(*t*) и *s*(*t*) на отрезке [-0,5; 0,5];
   c. Постройте на одном рисунке графики *f*(*t*) и *s*(*t*) на отрезке [-1;1];
   d. Постройте на одном рисунке графики *f*(*t*) и *s*(*t*) на отрезке [-4;4];
   e. Разложите функцию $f(t) = t^2 + 2sin(\pi \cdot t) + 4cos(3\pi \cdot t)$ в степенной ряд *s*(*t*) в точке x = 0, используя 5 членов разложения;
   f. Постройте на одном рисунке графики *f*(*t*) и *s*(*t*) на отрезке [-1; 1].
3. Ряд Фурье
   a Разложите функцию $f(t) = t^2 + 2sin(\pi \cdot t) + 4cos(3\pi \cdot t)$ в ряд Фурье *r*(*t*) на отрезке [-4; 4], используя 3 члена разложения
   b Постройте на одном рисунке графики *f*(*t*) и *r*(*t*) на отрезке [-4; 4];
   c Разложите функцию $f(t) = t^2 + 2sin(\pi \cdot t) + 4cos(3\pi \cdot t)$ в ряд Фурье *r*(*t*) на отрезке [-4; 4], используя 5 членов разложения
   d Постройте на одном рисунке графики *f*(*t*) и *s*(*t*) на отрезке [-4; 4]
4. Вычислите в аналитическом виде коэффициенты Фурье a_n и b_n для функции $f(t) = t^2 + 2sin(\pi \cdot t) + 4cos(3\pi \cdot t)$, используя то, что $sin(\pi n) = 0, cos(\pi n) = (-1)^n$и команду `simplify`.

2. Работа с матрицами и массивами. Библиотека NumPy

Для работы с библиотекой Numpy её надо загрузить

```
import numpy as np
```

Основной структурой библиотеки NumPy является массив. Вектор может быть представлен как одномерный массив в виде строки

```
vector_row = np.array([1,2,3,4,5,6])
```

или столбца

```
vector_column = np.array([[1],[2],[3],[4]]).
```

Матрица представляет собой двумерный массив.

```
matrix = np.array([[1,2],[3,4]])
print(matrix)
```

[[1 2]
[3 4]]

Как и для других объектов Python, индексация массивов в NumPy начинается с нуля. Для выбора элементов в NumPy предлагается много возможностей

```
vector_row[1] #элемент вектор-строки с индексом 1
```

2

```
vector_row[2:] #все элементы вектор-строки с индексами 2 и больше
```

array([3, 4, 5, 6])

```
vector_row[:3]  #все элементы вектор-строки с индексами меньше 3
```

array([1, 2, 3])

```
vector_row[-1]  #последний элемент вектор-строки
```

6

```
matrix[:,1:2]   #все строки, второй столбец матрицы
```

array([[2],
[4]])

```
matrix[0,:] #первая строка, все столбцы матрицы
```

array([1, 2])

Максимальное и минимальное значение в массиве можно найти, используя команды `max` и `min`. Если надо найти максимум или минимум в столбце или строке, следует использовать параметр `axis`

```
matrix1 = np.array([[2,3,4],[2,7,8],[3,9,0]])
print(np.max(matrix1)) #максимум матрицы
print(np.min(matrix1, axis = 0)) #минимум в каждом столбце
print(np.max(matrix1, axis = 1)) #максимум в каждой строке
```

9
[2 3 0]
[4 8 9]

Транспонировать матрицу можно, используя команду `T`

```
matrix1.T
```

```
array([[2, 2, 3],
       [3, 7, 9],
       [4, 8, 0]])
```

Но транспонировать векторы (вектор-строку в вектор столбец и наоборот) надо аккуратно.

```
np.array([1,2,3]).T
array([1, 2, 3])
np.array([[1,2,3]]).T
array([[1],
       [2],
       [3]])
```

Для вычисления определителя матрицы в NumPy используется команда `det` из модуля `linalg`

```
np.linalg.det(matrix1)
-83.99999999999999
```

Чтобы получить обратную матрицу, в NumPy используется команда `inv` из модуля `linalg`

```
np.linalg.inv(matrix1)
array([[0.85714286, -0.42857143,  0.04761905],
       [-0.28571429,  0.14285714,  0.0952381],
       [ 0.03571429,  0.10714286, -0.0952381]])
```

Для нахождения собственных значений и собственных векторов в NumPy используется команда `eig` из модуля `linalg`

```
eigenvals, eigenvecs = np.linalg.eig(matrix1)
print(eigenvals) #собственные значения
print(eigenvecs) #собственные векторы
[14.  1. -6.]
[[-3.71390676e-01 -9.48683298e-01 -2.40771706e-01]
 [-7.42781353e-01  3.16227766e-01 -4.81543412e-01]
 [-5.57086015e-01  1.65380959e-16  8.42700972e-01]]
```

Сложение и вычитание матриц одного и того же размера производится с использованием операторов + и -

```
matrix_a = np.array([[1,2,3],
                     [4,5,6]])
matrix_b = np.array([[2,-5,-1],
                     [3,8,-2]])
matrix_a + matrix_b
array([[ 3, -3,  2],
       [ 7, 13,  4]])
matrix_a - matrix_b
array([[-1,  7,  4],
       [ 1, -3,  8]])
```

Для поэлементного умножения матриц одного и того же размера используется оператор * (так же как и для скалярного произведения векторов)

```
matrix_a * matrix_b
array([[ 2, -10,  -3],
    [ 12,  40, -12]])
```

Для умножения матриц используется оператор @

```
matrix_c = np.array([[1,2],
                     [-3,4],
                     [5,-6]])
matrix_a @ matrix_c
array([[10, -8],
    [19, -8]])
matrix_c @ matrix_a
array([[ 9,  12,  15],
    [ 13,  14,  15],
    [-19, -20, -21]])
```

Для генерации случайных чисел следует использовать модуль `random` библиотеки NumPy.

Для получения одних и тех же случайных чисел при каждом запуске программы следует установить начальное значение генератора псевдослучайных чисел, например,

```
np.random.seed(4321)
```

Команда `random` генерирует случайные числа на интервале [0,1)

```
np.random.random(5)
array([0.07080288, 0.81506401, 0.76790496, 0.2863545, 0.19309431])
```

Команда `uniform(a,b,n)` генерирует n случайных чисел на интервале [0,1)

```
np.random.uniform(1,5,3)
array([4.91564884, 2.62491485, 4.03107147])
```

Команда `normal(a,b,n)` генерирует n случайных чисел с математическим ожиданием a и стандартным отклонением b

```
np.random.normal(1,3,3)
array([0.20569776, -0.71651392, -1.25334041])
```

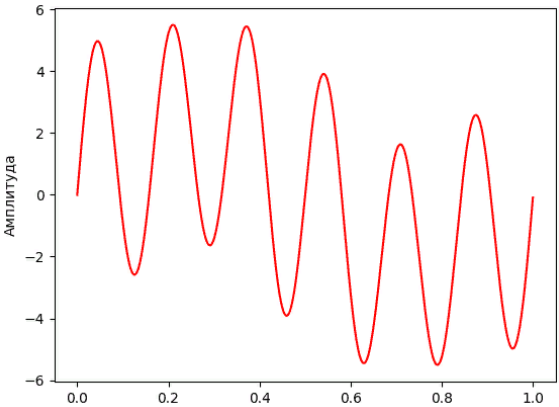
В пакете NumPy имеются модуль fft быстрого преобразования Фурье (версии дискретного) и соответствующие команды fft и ifft прямого и обратного преобразования.

```
from numpy.fft import fft, ifft
```

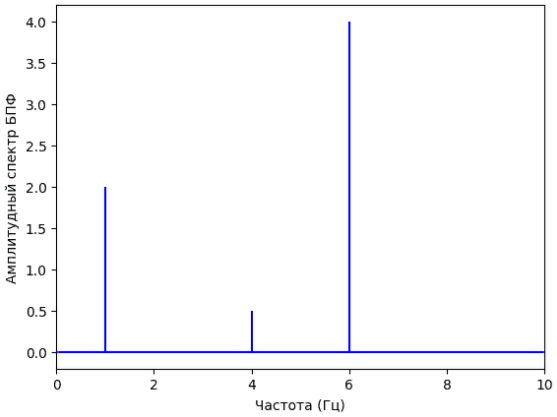
Пример выполнения преобразования Фурье и обратного преобразования Фурье приведён ниже. Заметим, что для построения использована библиотека `matplotlib`

```
sr = 2000 #частота дискретизации
ts = 1.0/sr #интервал выборки
t = np.arange(0,1,ts)
freq = 1.
x = 2*np.sin(2*np.pi*freq*t)
freq = 4
```

```
x = x + 0.5*np.sin(2*np.pi*freq*t)
freq = 6
x = x + 4* np.sin(2*np.pi*freq*t)
import matplotlib.pyplot as plt
plt.plot(t, x, 'r')
plt.ylabel('Amplitude')
plt.show()
```

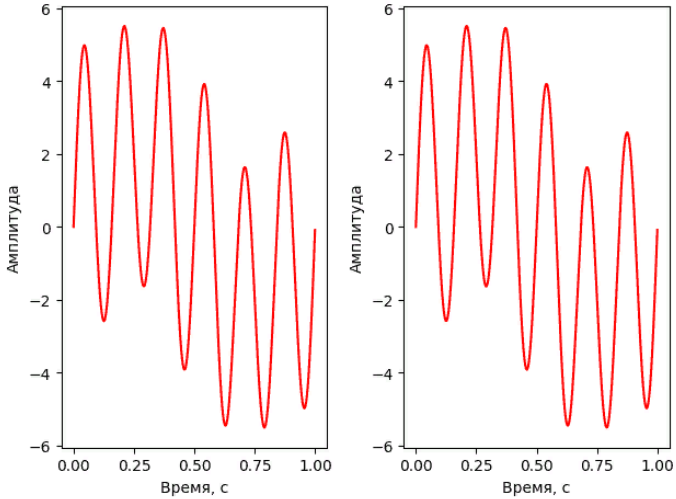



```
X = fft(x)
N = len(X)
n = np.arange(N)
T = N/sr
freq = n/T
plt.stem(freq,   np.abs(X)/(N//2),   'b',   markerfmt="   ",
basefmt="-b")
plt.xlabel('Freq (Hz)')
plt.ylabel('FFT Amplitude |X(freq)|')
plt.xlim(0, 10)
```



```
plt.subplot(121)
plt.plot(t, x, 'r')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')

plt.subplot(122)
plt.plot(t, ifft(X), 'r')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```



Задания.

1. Заданы матрицы $A = \begin{pmatrix} 4 & 1 & -2 \\ 3 & -2 & 5 \end{pmatrix}$ и $B = \begin{pmatrix} -1 & 3 \\ 2 & -3 \\ 4 & 1 \end{pmatrix}$
   a. найдите максимальные значения в каждом столбце матрицы *A*;
   b. найдите минимальные значения в каждой строке матрицы *B*;
   c. транспонируйте матрицы *A* и *B*;
   d. вычислите сумму $A + B^{\mathrm{T}}$ и разность $B - A^{\mathrm{T}}$;
   e. перемножьте поэлементно *A* и B^{T}, *B* и A^{T};
   f. найдите произведения матриц *AB, BA,* AA^T, $A^T A$;
   g. найдите собственные значения и собственные векторы матриц *AB, BA,* AA^T, $A^T A$.
2. Пусть имеется дискретный сигнал $x = f(t)$ с частотой дискретизации $sr = 1024$, заданный на интервале [0,1).
   a. постройте график функции $f(t)$;
   b. вычислите быстрое преобразование Фурье $X(\omega)$ функции $f(t)$;

c. постройте график амплитудного спектра функции $X(\omega)$;
d. вычислите обратное преобразование Фурье $f1(t)$ функции $X(\omega)$;
e. постройте рядом графики $f(t)$ и $f1(t)$, сравните их.

Функции $x = f(t)$ следующие:

1. $\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t)$;
2. $\cos(2\cdot\pi\cdot t)$, если $t \in [0,1/2)$,
   $4\cos(6\cdot\pi\cdot t)$; если $t \in [1/2,1)$;
3. $4\cos(6\cdot\pi\cdot t)$, если $t \in [0,1/2)$,
   $\cos(2\cdot\pi\cdot t)$; если $t \in [1/2,1)$;
4. $\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t) + r(t)$, где $r(t)$ - случайная компонента с равномерным распределением на отрезке [-0,5;0,5]
5. $\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t) + r(t)$, где $r(t)$ – нормально распределённая случайная компонента с математическим ожиданием 0 и стандартным отклонением 0,5;
6. $\exp(t)\cdot(\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t))$.

3. Вейвлеты. Библиотека PyWavelets

Для работы вейвлетами в Python разработана библиотека PyWavelets. Для работы с ней её надо загрузить (как обычно загружаются с библиотекой PyWavelets её надо загрузить (например, http://pypi.python.org/pypi/PyWavelets/ или https://github.com/PyWavelets/pywt), инсталлировать и подключить

```
import pywt
```

Чтобы ознакомиться с семействами имеющихся вейвлетов, можно использовать команду `families()`, а для того, чтобы узнать, какие конкретные вейвлеты реализованы в Python, используется команда `wavelist()` с параметром kind = 'all', 'continuous' или 'discrete'.

Свойства конкретного вейвлета можно получить следующим образом:

```
w1 = pywt.ContinuousWavelet('mexh')
print(w1)
```

```
ContinuousWavelet mexh
 Family name:   Mexican hat wavelet
 Short name:    mexh
 Symmetry:      symmetric
 DWT:           False
 CWT:           True
 Complex CWT:   False
```

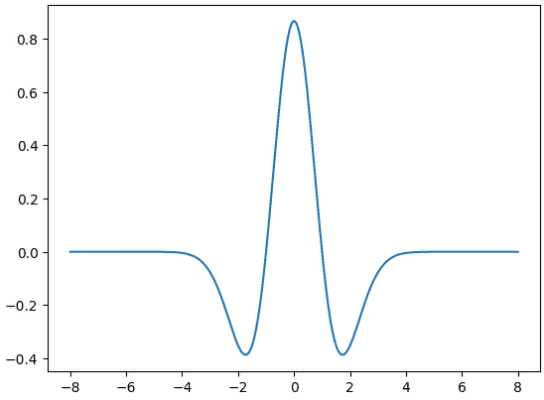
или

```
w2 = pywt.ContinuousWavelet('db2')
print(w2)
```

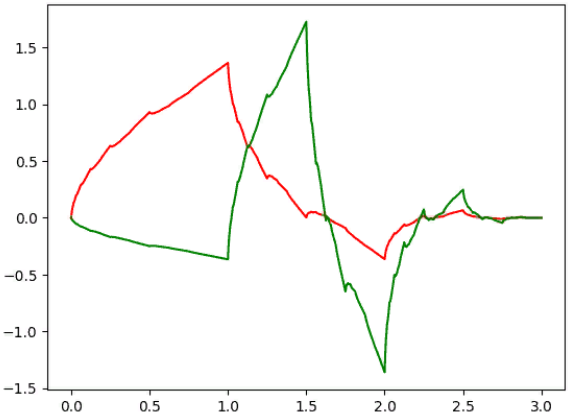
```
Wavelet db2
 Family name:   Daubechies
 Short name:    db
 Filters length: 4
 Orthogonal:    True
 Biorthogonal:  True
 Symmetry:      asymmetric
 DWT:           True
 CWT:           False
```

Для непрерывных вейвлетов можно построить график самого вейвлета, для дискретных — график скейлинг-функции и вейвлета

```
psi, x = w1.wavefun()
plt.plot(x,psi)
```




```
phi, psi, x = w2.wavefun()
plt.plot(x, phi, 'r')
plt.plot(x, psi, 'g')
```



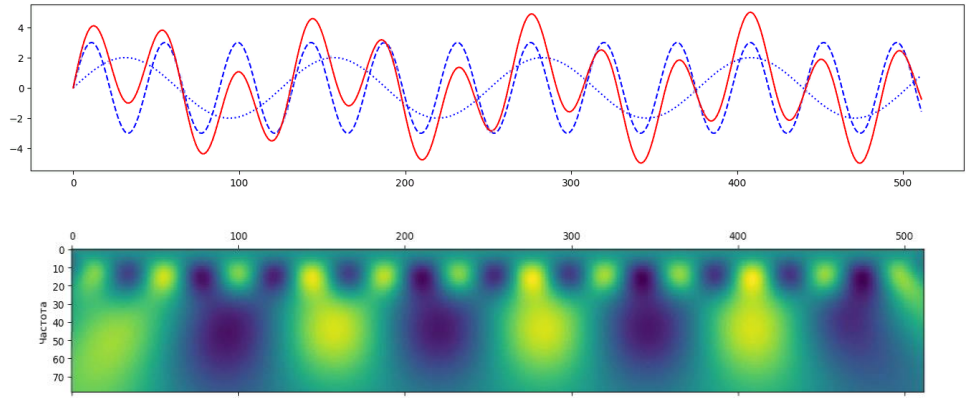
Для построения графиков была подключена библиотека `matplotlib`.

Непрерывные вейвлеты применяются для анализа сигналов, нахождения точек экстремума, разрывов, временно-частотного анализа и т. д.

Рассмотрим сумму двух синусоид, к которой применено непрерывное вейвлет-преобразование с семейством вейвлетов «мексиканская шляпа».

```
x = np.arange(512)
y1 = 2* np.sin(2*np.pi*x/125.6)
y2 = 3* np.sin(2*np.pi*x/44.1)
y = y1 +y2
coef, freqs=pywt.cwt(y,np.arange(1,80),'mexh')
plt.plot(x, y1, 'b:')
plt.plot(x, y2, 'b--')
plt.plot(x, y, 'r')
```

```
plt.matshow(coef)
plt.ylabel('Частота')
plt.show()
```



Задания.

1. Для каждого из перечисленных ниже семейств выберите по одному вейвлету, укажите его свойства, постройте порождающий вейвлет, а для дискретных вейвлетов и скейлинг-функцию.
   Семейства:
   'Haar', 'Daubechies', 'Symlets', 'Coiflets', 'Biorthogonal', 'Reverse biorthogonal', 'Discrete Meyer (FIR Approximation)', 'Gaussian', 'Morlet wavelet', 'Shannon wavelets', 'Frequency B-Spline wavelets'.
2. Пусть имеется дискретный сигнал $x = f(t)$ с частотой дискретизации $sr = 2048$, заданный на интервале [0,1)
   a. постройте график функции $f(t)$;
   b. вычислите непрерывное вейвлет-преобразование $W(a,b)$ функции $f(t)$ (порождающий вейвлет определяется преподавателем);
   c. постройте график функции $W(a,b)$.

   Функции $x = f(t)$ следующие:
   1. $\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t)$;
   2. $\cos(2\cdot\pi\cdot t)$, если $t \in [0,1/2)$,
      $4\cos(6\cdot\pi\cdot t)$; если $t \in [1/2,1)$;
   3. $4\cos(6\cdot\pi\cdot t)$, если $t \in [0,1/2)$,
      $\cos(2\cdot\pi\cdot t)$; если $t \in [1/2,1)$;
   4. $\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t) + r(t)$, где $r(t)$ - случайная компонента с равномерным распределением на отрезке [-0,5;0,5]
   5. $\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t) + r(t)$, где $r(t)$ – нормально распределённая случайная компонента с математическим ожиданием 0 и стандартным отклонением 0,5;
   6. $\exp(t)\cdot(\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t))$.

4. Дискретное вейвлет-преобразование

Чтобы выполнить дискретное вейвлет-преобразование сигнала, сначала следует определить, сколько уровней разложения будет использоваться.

Если сигнал раскладывается только на аппроксимирующую и детализирующую компоненты, то можно применить команду `dwt(данные, вейвлет)` библиотеки `pywt`

```
import numpy as np
import matplotlib.pyplot as plt
import pywt
x = np.arange(512)
y1 = 2* np.sin(2*np.pi*x/125.6)
y2 = 3* np.sin(2*np.pi*x/44.1)
y = y1 +y2
coeffs = pywt.dwt([y], 'db1')
```

Данные `coeffs` – это список двух массивов, в первом из которых хранятся аппроксимирующие коэффициенты (АК), а во втором — детализирующие (ДК).

```
CA = coeffs[0][0,:]
CD = coeffs[1][0,:]
```

Если построить графики этих коэффициентов

```
plt.subplot(2,1,1)
plt.plot(t,CA,'r')
plt.subplot(2,1,2)
plt.plot(t,CD,'b')
```

то можно увидеть, что детализирующие коэффициенты по сравнению с аппроксимирующими достаточно малы. Если их обнулить,

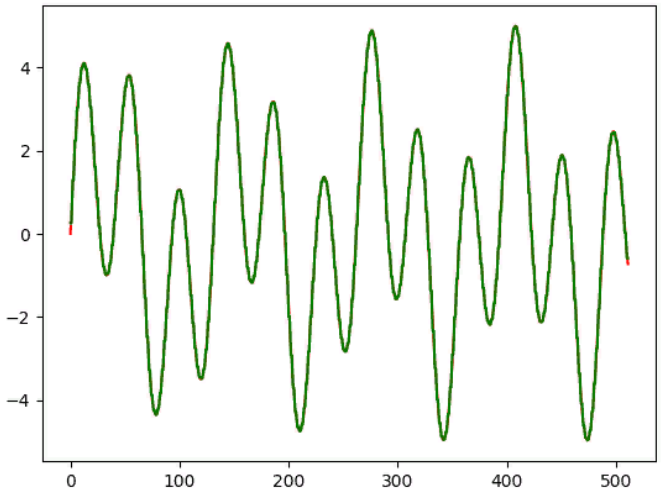
```
for i in range (0, len(CD)):
  CD[i] = 0
```

провести восстановление сигнала, используя команду `idwt(АК, ДК вейвлет)` (вейвлет должен быть тем же самым, что и для разложения) библиотеки `pywt`,

```
y1 = pywt.idwt(CA, CD, 'db1')
```

построить графики исходного и восстановленного сигнала, то можно увидеть, что различие между исходным и восстановленным сигналом достаточно мало, хотя использовалась лишь половина сигналов.

```
plt.plot(x,y,'r')
plt.plot(x,y,'g')
```

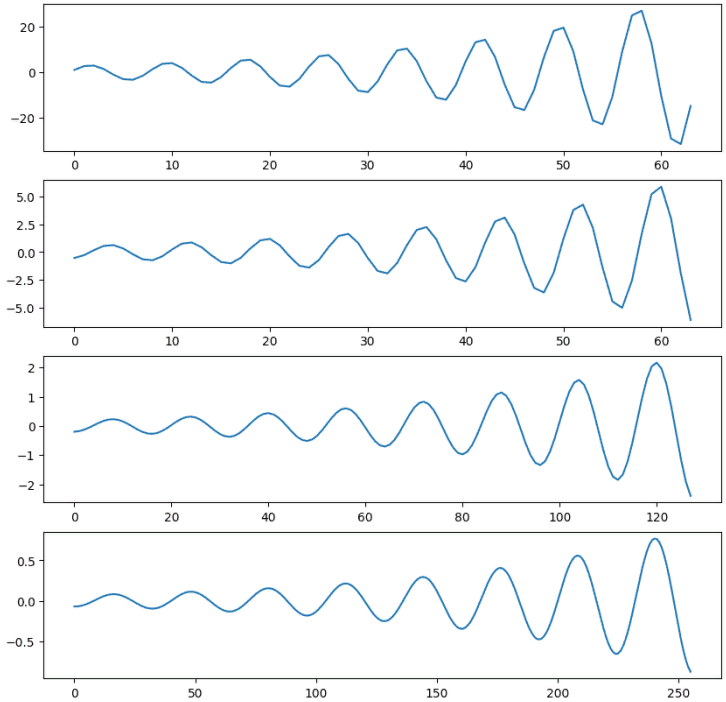


Если используется несколько уровней разложения, то можно либо поочерёдно несколько раз применить команду `dwt` к аппроксимирующим коэффициентам, либо воспользоваться командой `wavedec(данные, вейвлет, уровень = ...)`. Результатом выполнения команды будет список k массивов, где k на единицу больше числа уровней разложения.

```
x = np.arange(512)
y = np.exp(0.005*x)*np.sin(2*np.pi*x/64)
lev = 3
coeffs = pywt.wavedec([y], 'db4', level = lev )
cA, cD3, cD2, cD1 = coeffs
```

Построим графики коэффициентов

```
CA = cA[0,:]
CD3 = cD3[0,:]
CD2 = cD2[0,:]
CD1 = cD1[0,:]
t1 = np.arange(0,len(CA),1)
t2 = np.arange(0,len(CD2),1)
t3 = np.arange(0,len(CD1),1)
plt.subplot(411)
plt.plot(t1,CA)
plt.subplot(412)
plt.plot(t1,CD3)
plt.subplot(413)
plt.plot(t2,CD2)
plt.subplot(414)
plt.plot(t3,CD1)
plt.show()
```



Восстановление сигнала происходит с использованием команды `waverec(коэффициенты, вейвлет = …)`. В результате получаем кортеж.

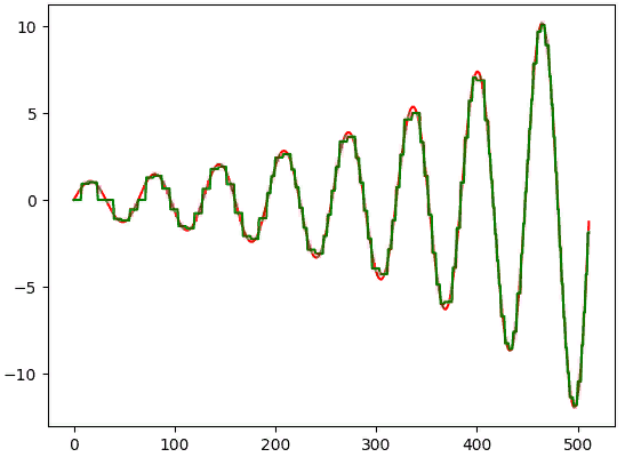
```
w1 = pywt.waverec(coeffs, 'db4')
```

Для возможности построения преобразуем данные из кортежа в массив.

```
w1 = w1[0,:]
```

Если оставить, например, 10 процентов коэффициентов, наибольших по абсолютной величине, а остальные обнулить, то можно восстановить сигнал с небольшой погрешностью.

```
plt.plot(y, 'r')
plt.plot(w1, 'g')
plt.show()
```



Задания.

1. Примените одноуровневое дискретное вейвлет-преобразование к следующим функциям, заданным на интервале [0,4) с 1024 точками отсчёта
   a cos(2·π·*t*) + 4 cos(6·π·*t*) + *r*(*t*), где *r*(*t*) - случайная компонента с равномерным распределением на отрезке [-0,5;0,5]
   b 4·*t* - t^2 + *r*(*t*), где *r*(*t*) – нормально распределённая случайная компонента с математическим ожиданием 0 и стандартным отклонением 0,5;
   c *t* + exp(0,3·*t*)·(4 cos(6·π·*t*)).

   Для каждой функции
   - Обнулите детализирующие коэффициенты, восстановите сигнал.
   - Найдите максимальную погрешность восстановления.
   - Постройте графики исходного и восстановленного сигнала.
   - Используйте следующие вейвлеты: haar, db4, dmey,bior2.2, rbio2.2, sym4.
   - Для каждой из исходной функции укажите наилучший вейвлет для вейвлет-преобразования.
2. Пусть имеется дискретный сигнал *x* = $f_1(t)$ + $f_2(t)$ с частотой дискретизации *sr* = 1024, заданный на интервале [0,4), где
   $f_1(t) = t^2 + \sin(2\cdot\pi\cdot t)$ на интервале [0,2)
   $f_2(t) = 4 - t + \exp(-0{,}2\cdot t)\cdot\sin(4\cdot\pi\cdot t)$ на интервале [2,4)
   a. Положите число уровней разложения 3
   b. Проведите разложение сигнала, выбрав вейвлет db4
   c. Постройте графики коэффициентов
   d. Оставьте 10% наибольших по абсолютной величине коэффициентов
   e. Восстановите сигнал

f. Найдите суммарную по всем точкам погрешность восстановления
g. Постройте на одном рисунке графики исходного и восстановленного сигнала
h. Повторите пункты b – g, при числе уровней 5
i. Повторите пункты b – g, при числе уровней 10
j. Выберите в качестве вейвлета bior 2.2 и повторите вычисления с числом уровней 3, 5, 10
k. Выберите в качестве вейвлета sym4 и повторите вычисления с числом уровней 3, 5, 10
l. Проанализируйте полученные результаты

5. Вейвлет-преобразования изображений

Для работы с изображениями необходимо подключить несколько библиотек, кроме уже использовавшихся `numpy`, `pywavelets`, `matplotlib`.

```
import numpy as np
import pywt
import matplotlib.pyplot as plt
```

Для загрузки файла изображения используется, например, библиотеки `PIL` или `skimage.io`

```
from PIL import Image
import skimage.io
```

Для устранения шума в сигнале понадобятся различные пакеты библиотеки `skimage`

```
from skimage.restoration import (denoise_wavelet,
estimate_sigma)
from skimage.util import random_noise
from skimage.metrics import peak_signal_noise_ratio
```

Дискретное вейвлет-преобразование изображения аналогично такому же преобразованию сигнала (заметим, что изображение должно быть представлено как массив).

Если предполагается один уровень разложения исходного изображения, то можно применить команду `dwt2(изображение, вейвлет)` библиотеки `pywt`.

```
input_image = Image.open('NTNU.jpg').convert('L')
image_array = np.array(input_image)
coeffs2 = pywt.dwt2(image_array, 'haar')
cA,(cH, cV, cD) = coeffs2
```

Данные `coeffs2` – это список массива и состоящего из трёх массивов кортежа. В первом массиве хранятся аппроксимирующие коэффициенты, а в массивах кортежа поочерёдно горизонтальные детализирующие, вертикальные детализирующие и диагональные детализирующие коэффициенты.

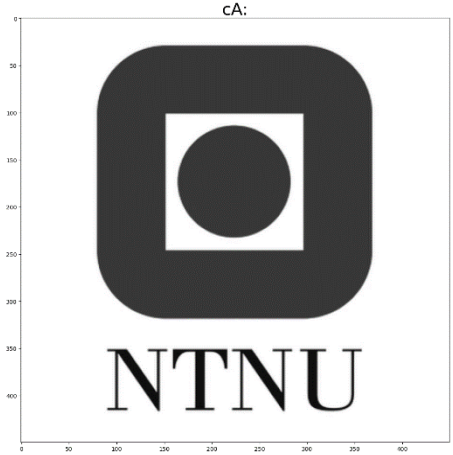
Если построить изображения этих коэффициентов на одном рисунке

```
plt.figure(figsize = (30,30))
plt.subplot(2,2,1)
plt.imshow(cA,cmap =plt.cm.gray)
plt.title('cA', fontsize = 30)
plt.subplot(2,2,2)
plt.imshow(cH,cmap =plt.cm.gray)
plt.title('cH', fontsize = 30)
plt.subplot(2,2,3)
plt.imshow(cV,cmap =plt.cm.gray)
plt.title('cV', fontsize = 30)
plt.subplot(2,2,4)
```

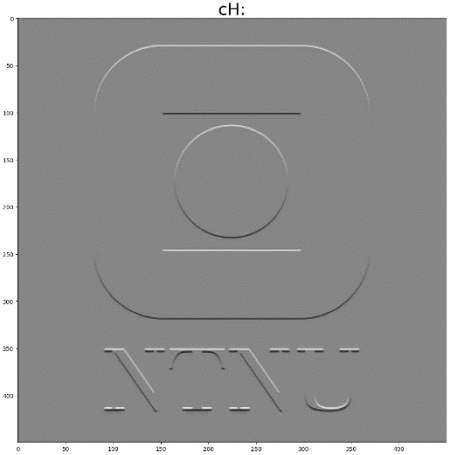
```
plt.imshow(cD,cmap =plt.cm.gray)
plt.title('cD', fontsize = 30)
```

то можно увидеть особенности исходного изображения в соответствующих направлениях.

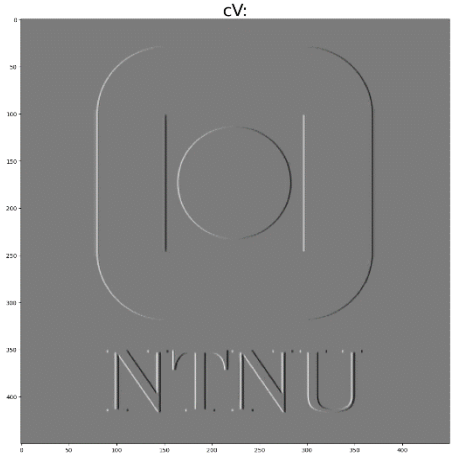
cA:



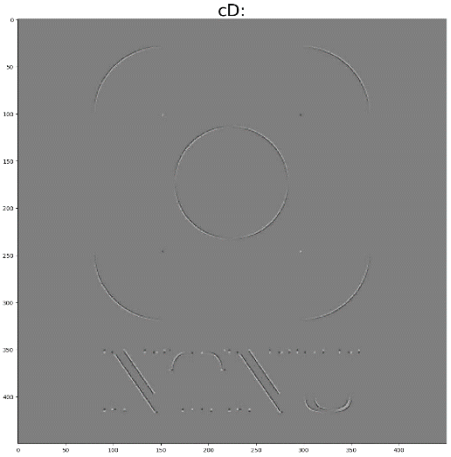
cH:



cV:

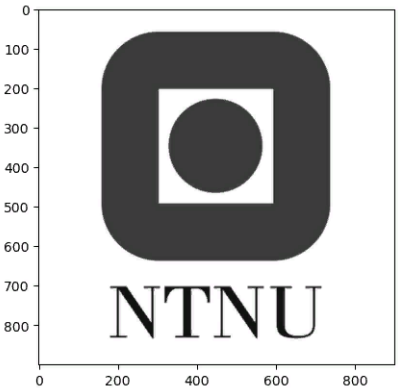


cD:



Чтобы провести восстановление сигнала, используется команда `idwt2(коэффициенты, вейвлет)` (вейвлет должен быть тем же самым, что и для разложения) библиотеки `pywt`,

```
output_image = pywt.idwt2(coeffs2 'haar')
plt.imshow(output_image,cmap =plt.cm.gray)
plt.show()
```

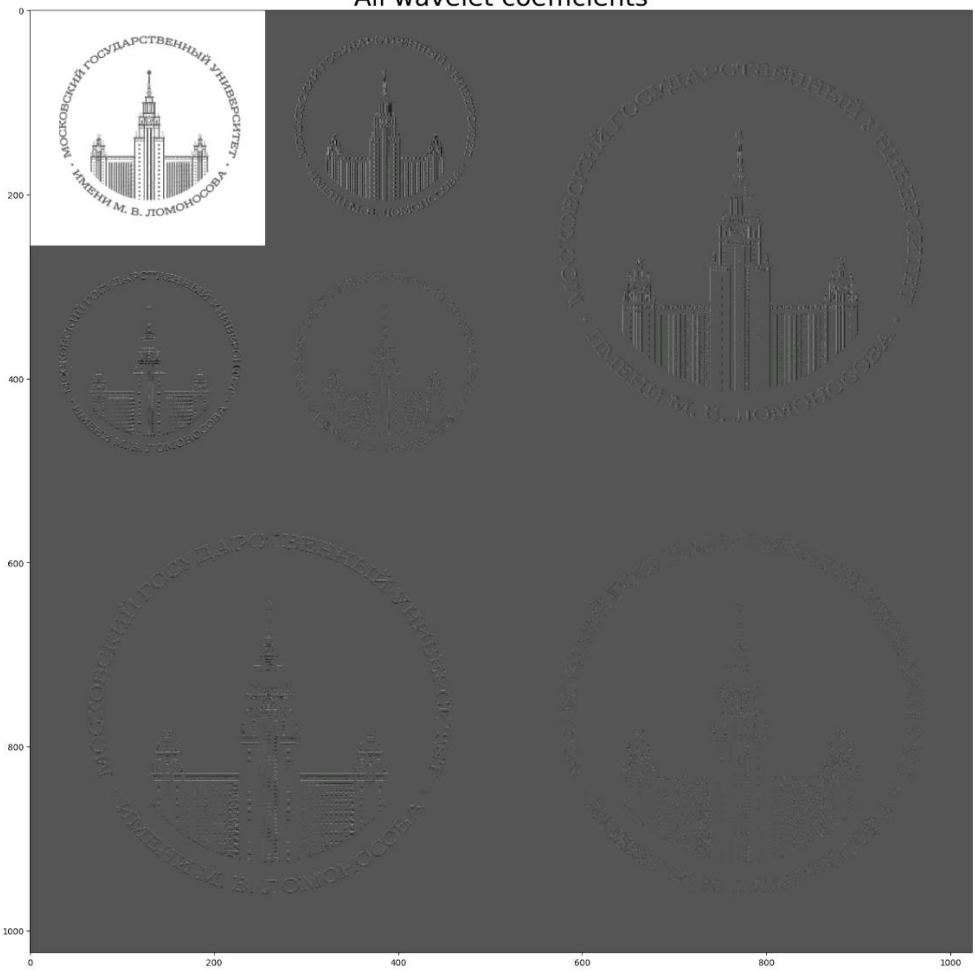


Если используется несколько уровней разложения, то можно либо поочерёдно несколько раз применить команду `pywt.dwt2` к аппроксимирующим коэффициентам, либо воспользоваться командой `wavedec2(данные, вейвлет, уровень = ...)`. Результатом выполнения команды будет список из массива аппроксимирующих коэффициентов и кортежей детализирующих.

```
img = skimage.io.imread('mgu.bmp')
img = skimage.img_as_float(img)
C = pywt.wavedec2(img,'db4',level= 2)
cA2 = C[0]
(cH1,cV1,cD1) = C[-1]
(cH2,cV2,cD2) = C[-2]
```

Можно построить изображения аппроксимирующих и детализирующих коэффициентов.

```
arr, coeff_slices = pywt.coeffs_to_array(C)
plt.figure(figsize = (20,20))
plt.imshow(arr,cmap =plt.cm.gray)
plt.title('All wavelet coefficients', fontsize = 30)
```



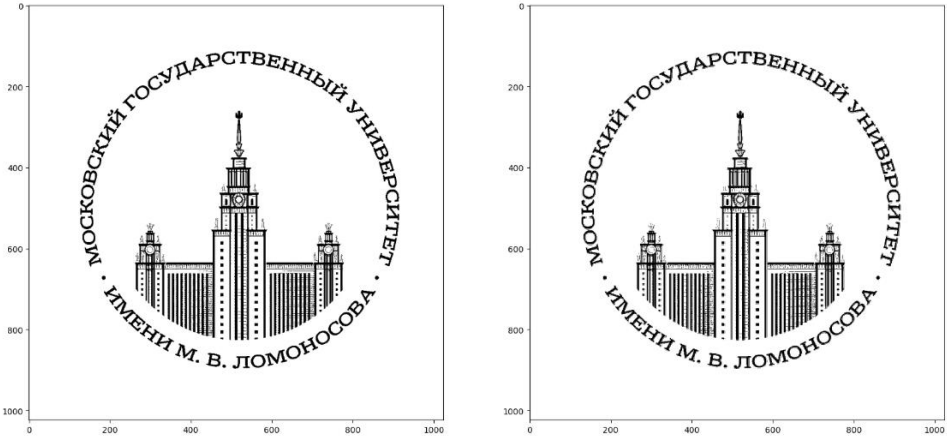
Восстановление сигнала происходит с использованием команды `waverec2(коэффициенты, вейвлет = …)`.

```
w1 = pywt.waverec2(C, 'db4')
```

Для сжатия сигнала можно либо самостоятельно оставлять необходимое число коэффициентов, либо воспользоваться встроенными функциями

```
input_image = Image.open('mgu.bmp').convert('L')
image_array = np.array(input_image)
coeffs2 = pywt.dwt2(image_array, 'db1')
cA,(cH, cV, cD) = coeffs2
compression_ratio = 0.2
```

```
compressed_coeffs = tuple(coef * (pywt.threshold(coef, value=(compression_ratio * np.max(coef)), mode="soft")) for coef in coeffs2)
compressed_image_array = pywt.idwt2(compressed_coeffs,'db1')
compressed_image_array = np.asarray(compressed_image_array)
compressed_image = Image.fromarray(compressed_image_array)
plt.figure(figsize = (20,20))
plt.subplot(2,2,1)
plt.imshow(input_image,cmap =plt.cm.gray)
plt.subplot(2,2,2)
plt.imshow(compressed_image,cmap =plt.cm.gray)
```



По рисунку видно, что, оставляя всего 20% коэффициентов и применяя «мягкий» порог обнуления, изображение восстанавливается достаточно хорошо.

Для выбора лучшего вейвлета можно рассмотреть разницу между исходным и восстановленным изображениями.

```
err1 = image_array-compressed_image_array
np.linalg.norm(err1)
```

101429912.4

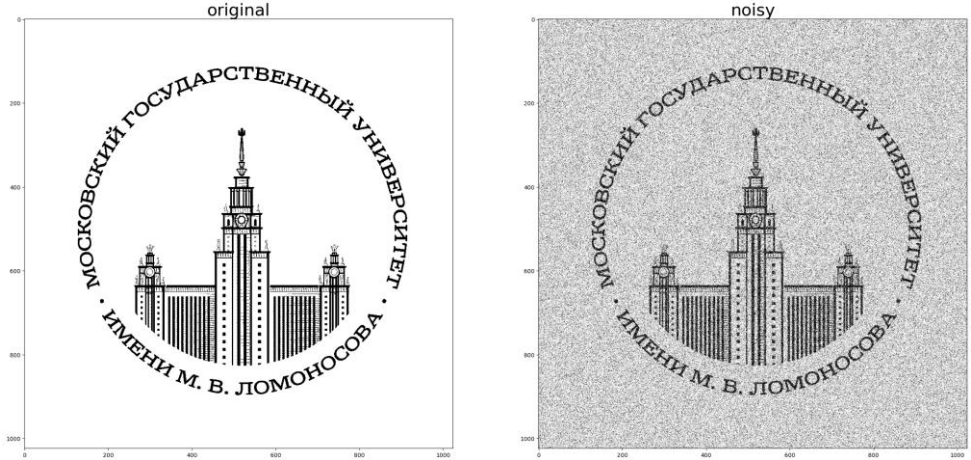
Для устранения шума можно также воспользоваться встроенными функциями. Импортируем некоторые функции библиотеки skimage.io

```
from skimage.restoration import (denoise_wavelet, estimate_sigma)
from skimage.util import random_noise
from skimage.metrics import peak_signal_noise_ratio
```

Наложим случайным белый (гауссовский) шум с среднеквадратичным отклонением 0,5 на изображение

```
sigma = 0.5
imgn = random_noise(img,var=sigma**2)
sigma_est =estimate_sigma(img,average_sigmas = True)
```

```
plt.figure(figsize = (30,30))
plt.subplot(1,2,1)
plt.imshow(img,cmap =plt.cm.gray)
plt.title('original', fontsize = 30)
plt.subplot(1,2,2)
plt.imshow(imgn,cmap =plt.cm.gray)
plt.title('noisy', fontsize = 30)
plt.show()
```



Уменьшим шум, используя функции `denoise_wavelet`

Для Python пока реализованы два метода устранения – универсальный (`Visu`) и байесовский (`Bayes`).

Перед применением метода `Visu` следует предварительно оценить дисперсию

```
sigma_est = estimate_signal(),
```

а затем устранить шум

```
y = denoise_wavelet(x,wavelet = 'Wname', mode = 'soft/hard',
wavelet_levels = n, method = 'VisuShrink',sigma = sigma_est,
rescale_sigma = 'True/False')
```

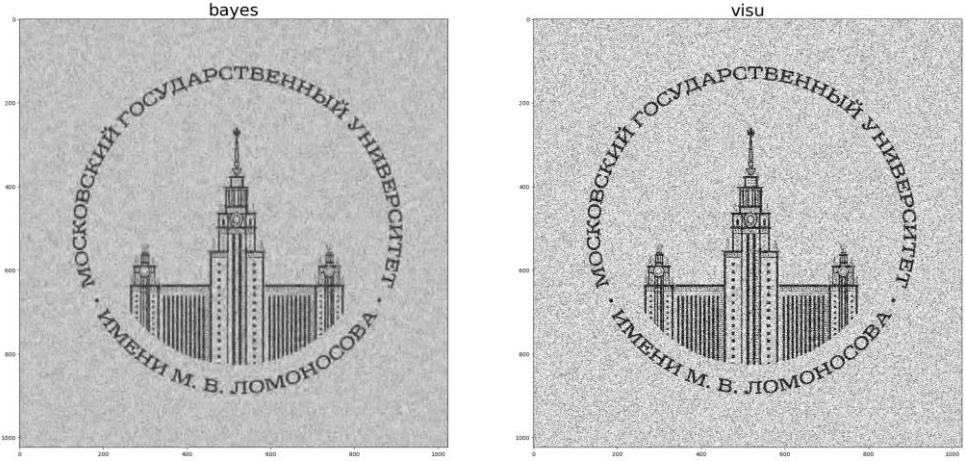
Здесь `x` – входящее изображение, `y` – выходное изображение, `Wname` – используемый вейвлет, `soft/hard` – мягкий или жёсткий порог обнуления, `n` – число уровней разложения (по умолчанию – максимальное), `True/False` – пересчёт среднеквадратичного отклонения.

У байесовского метода параметров меньше

```
y = denoise_wavelet(x,wavelet = 'Wname', mode = 'soft/hard',
wavelet_levels = n, method =  'BayesShrink', rescale_sigma =
'True/False')
plt.figure(figsize = (30,30))
plt.subplot(1,2,1)
plt.imshow(img_bayes,cmap =plt.cm.gray)
```

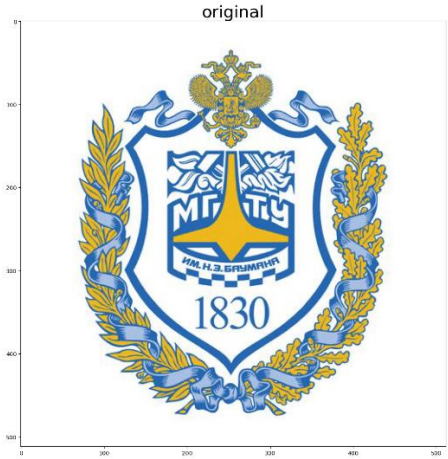
```
plt.title('bayes', fontsize = 30)

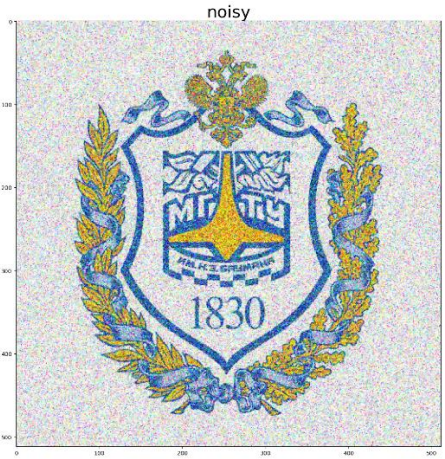
plt.subplot(1,2,2)
plt.imshow(img_visu,cmap =plt.cm.gray)
plt.title('visu', fontsize = 30)
plt.show()
```



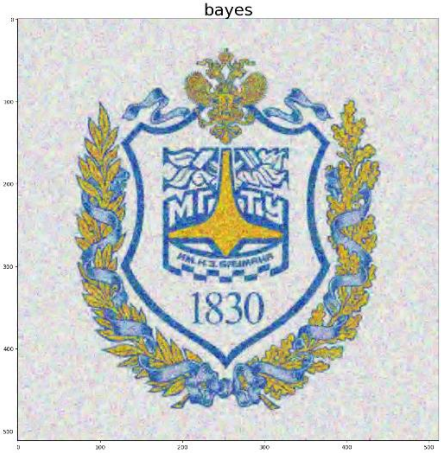
Если требуется уменьшить шум в цветном изображении, то можно добавить ещё два параметра: `channel_axis=-1, convert2ycbcr=True` для того, чтобы перекодировать изображение (кодируется каждый канал по отдельности) из цветового пространства RGB в пространство YCbCr, что, как правило, даёт лучший результат.

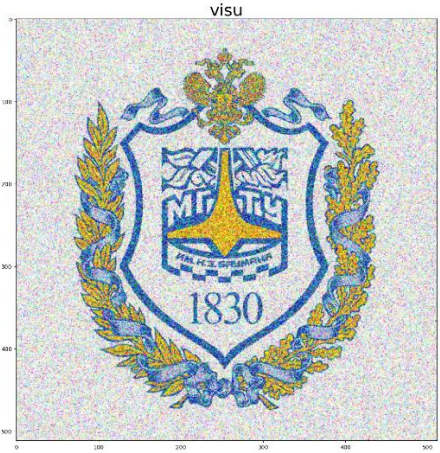
```
img = skimage.io.imread('bmstu2.jpg')
img = skimage.img_as_float(img)
sigma = 0.3
imgn = random_noise(img,var=sigma**2)
sigma_est = estimate_sigma(img, channel_axis=-1,
average_sigmas = True)
plt.figure(figsize = (30,20))
plt.subplot(1,2,1)
plt.imshow(img,cmap =plt.cm.gray)
plt.title('original', fontsize = 30)
plt.subplot(1,2,2)
plt.imshow(imgn,cmap =plt.cm.gray)
plt.title('noisy', fontsize = 30)
plt.show()
```






```
img_bayes = denoise_wavelet(imgn,channel_axis=-1,
convert2ycbcr=True, method='BayesShrink', wavelet = 'sym4',
mode='soft', rescale_sigma=True)
img_visu = denoise_wavelet(imgn, channel_axis=-1,
convert2ycbcr=True, method='VisuShrink', wavelet = 'sym4',
mode='soft', sigma=sigma_est/3, rescale_sigma=True)
plt.figure(figsize = (30,20))
plt.subplot(1,2,1)
plt.imshow(img_bayes,cmap =plt.cm.gray)
plt.title('bayes', fontsize = 30)
plt.subplot(1,2,2)
plt.imshow(img_visu,cmap =plt.cm.gray)
plt.title('visu', fontsize = 30)
plt.show()
```





Исходя из соотношения сигнал/шум, видим, что байесовский метод даёт значительное улучшение изображения

```
psnr_noisy = peak_signal_noise_ratio(img,imgn)
psnr_bayes = peak_signal_noise_ratio(img,img_bayes)
psnr_visu = peak_signal_noise_ratio(img,img_visu)
print(psnr_noisy)
print(psnr_bayes)
print(psnr_visu)
```

12.95045
17.38647
12.95045

Задания.

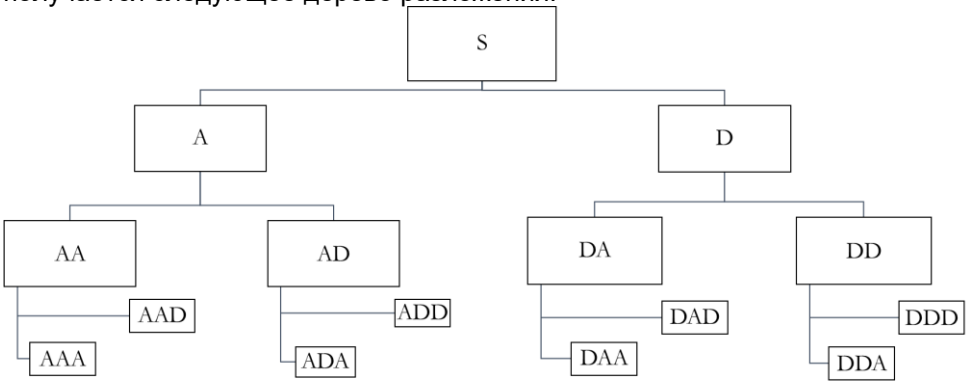
1. Сжатие изображений
   a. Загрузите изображение размера 256×256, 512×512 или 1024×1024 (лучше черно-белое формата bmp), используя шкалу и цветовую карту gray.
   b. Проведите одноуровневое разложение, используя вейвлет Хаара.
   c. Проведите сжатие изображения с коэффициентом сжатия 0,5 и восстановите его
   d. Постройте оригинальное и сжатое изображение на одном рисунке
   e. Найдите норму разницы между исходным и восстановленным изображением
   f. Повторите пункты a) – d), используя вейвлет Добеши 4, симметричные 4 и биортогональные 4,4 вейвлеты.
   g. Сделайте вывод, какой из вейвлетов лучший при одноуровневом разложении
   h. Проведите многоуровневое разложение (2-3 уровня), используя вейвлет Хаара.
   i. Проведите сжатие изображения с коэффициентом сжатия 0,5 (или оставляя 50%) коэффициентов и восстановите его
   j. Постройте оригинальное и сжатое изображение на одном рисунке
   k. Найдите норму разницы между исходным и восстановленным изображением
   l. Повторите пункты g) – k), используя вейвлет Добеши 4, симметричные 4 и биортогональные 4,4 вейвлеты.
   m. Сделайте вывод, какой из вейвлетов лучший при многоуровневом разложении

2. Сжатие изображений
   Используя изображение из пункта 1
   a. Наложите на него белый шум со стандартным отклонением σ = 0,5
   b. Найдите соотношение сигнал/шум до устранения.

c. Постройте исходное и зашумлённое изображения
d. Проведите устранение шума, используя вейвлет Хаара, порог – мягкий, метод – байесовский; рассчитайте соотношение сигнал/шум
e. Проведите устранение шума, используя вейвлет Хаара, порог – жёсткий, метод – байесовский; рассчитайте соотношение сигнал/шум
f. Определите, какой порог лучше
g. Проведите устранение шума, используя вейвлет Хаара, порог – мягкий, метод – универсальный; рассчитайте соотношение сигнал/шум
h. Проведите устранение шума, используя вейвлет Хаара, порог – жёсткий, метод – универсальный; рассчитайте соотношение сигнал/шум
i. Определите, какой порог лучше
j. Проведите устранение шума, используя вейвлет Хаара, порог – лучший (см. предыдущее задание), метод – универсальный, sigma = sigma_est; рассчитайте соотношение сигнал/шум
k. Проведите устранение шума, используя вейвлет Хаара, порог – лучший, метод – универсальный; sigma = sigma_est/3; рассчитайте соотношение сигнал/шум
l. Определите, какое значение sigma лучше использовать
m. Основываясь на результатах пунктов f и l, определите лучший метод и характеристики для устранения шумов
n. Используя этот метод и эти характеристики, устраните шум, используя вейвлеты Добеши 4, симметричные 4 и биортогональные вейвлеты 4.4, находя для каждого соотношение сигнал/шум
o. Составьте таблицу, включив в неё соотношения сигнал/шум, выберите наилучший вейвлет для устранения шума

6. Вейвлет-пакеты

В отличие от вейвлет-разложения сигналов и изображений при использовании вейвлет-пакетов раскладываются не только аппроксимирующие, но и детализирующие коэффициенты. В результате получается следующее дерево разложения:



Для работы с вейвлет-пакетами в Python имеются функции WaveletPacket

```
import pywt
x = [1, 2, 3, 4, 5, 6, 7, 8]
wp = pywt.WaveletPacket(data=x, wavelet='db1',
mode='symmetric')
```

Входные данные и коэффициенты разложения хранятся в структуре WaveletPacket.data. Узлы дерева идентифицируются путями. Для корневой вершины путь ' ', а уровень разложения — 0.

```
print(repr(wp.path))
print(wp.level)
```

Для проверки максимального уровня используется команда

```
print(wp.maxlevel)
```

Каждый узел дерева имеет несколько атрибутов: данные, путь, имя узла, материнская вершина, уровень, максимальный уровень и моду (a – аппроксимация, d — детализация)

```
print(wp['da'].data)
print(wp['da'].path)
print(wp['da'].node_name)
print(wp['da'].parent.path)
print(wp['da'].level)
print(wp['da'].maxlevel)
print(wp['da'].mode)
```

Можно записать все узлы в естественном порядке

```
print([node.path for node in wp.get_level(3, 'natural')])
print([node.data for node in wp.get_level(3, 'natural')])
```

Создадим новый вейвлет пакет и в некоторых из узлов введём данные

```
new_wp = pywt.WaveletPacket(data=None, wavelet='db1',
mode='symmetric')
new_wp['aa'] = wp['aa'].data
new_wp['ad'] = [-2., -2.]
new_wp['d'] = wp['d']
```

Восстановим сигнал по этому пакету

```
print(new_wp.reconstruct(update=False))
```

В этом случае мы получим исходный сигнал. Если же положить

```
new_wp['ad'] = [-1., -1.]
```

то восстановленный сигнал изменится.

```
print(new_wp.data)
```

Если значение update=False, то данные в узлах не меняются, если же положить update=True, то значения обновятся.

Можно удалить некоторые из узлов и восстановить сигнал по оставшимся.

```
wp2 = pywt.WaveletPacket(data=x, wavelet='db1',
mode='symmetric')
del wp2['ad']
del wp2['daa']
print(wp2.reconstruct())
```

Аналогично можно использовать вейвлет-пакеты для изображений, используя структуру Wavelet2D и помня, что имеются аппроксимирующие (a), горизонтальные (h), вертикальные (v) и диагональные (d) коэффициенты разложения.

Задания.

1. Пусть имеется дискретный сигнал $x = \cos(2 \cdot \pi \cdot t) + 2 \cos(6 \cdot \pi \cdot t)$ с частотой дискретизации $sr = 16$, заданный на интервале [0,1)
   m. проведите разложение сигнала в вейвлет-пакет, используя вейвлет Хаара;
   n. постройте дерево разложения;
   o. выведите все обозначения и все коэффициенты всех уровней разложения;
   p. удалите все листья максимального уровня разложения, абсолютные значения которых меньше 0,5, восстановите сигнал. Посчитайте, сколько ненулевых значений использовано. На одном рисунке постройте график исходного сигнала и график восстановленного сигнала, вычислите норму L_1 разности между сигналами;

q. Обнулите во всех массивах третьего уровня разложения коэффициенты, абсолютные значения которых меньше 0,5. Восстановите сигнал, используя только наборы третьего уровня разложения. Посчитайте, сколько ненулевых значений использовано. На одном рисунке постройте график исходного сигнала и график восстановленного сигнала, вычислите норму L_1 разности между сигналами;

r. Обнулите во всех массивах второго уровня разложения коэффициенты, абсолютные значения которых меньше 0,5. Восстановите сигнал, используя только наборы второго уровня разложения. Посчитайте, сколько ненулевых значений использовано. На одном рисунке постройте график исходного сигнала и график восстановленного сигнала, вычислите норму L_1 разности между сигналами;

s. Обнулите во всех массивах первого уровня разложения коэффициенты, абсолютные значения которых меньше 0,5. Восстановите сигнал, используя только наборы третьего уровня разложения. Посчитайте, сколько ненулевых значений использовано. На одном рисунке постройте график исходного сигнала и график восстановленного сигнала, вычислите норму L_1 разности между сигналами.

Выберите лучший (с наименьшим числом ненулевых коэффициентов) уровень разложения.

7. Вейвлет-пакеты и изображения

Вейвлет-пакеты можно использовать и для детального анализа сигналов и для изображений. Рассмотрим на конкретных примерах.

Сгенерируем сигнал

```
import numpy as np
import matplotlib.pyplot as plt
import pywt
sr = 64 #частота дискретизации
ts = 1.0/sr #интервал выборки
t = np.arange(0,1,ts)
freq = 1.
x = np.sin(2*np.pi*freq*t)
freq = 4
x = x + np.sin(2*np.pi*freq*t)
x1 = np.random.normal(0,0.2,sr)
x = x + x1
```

Проведём разложение сигнала на четыре уровня с использованием вейвлет-пакета. Опишем уровни разложения и выведем их обозначения на каждом уровне

```
wp = pywt.WaveletPacket(data=x,  wavelet='db1',
mode='symmetric', maxlevel=4)
print([node.path for node in wp.get_level(1, 'natural')])
print([node.path for node in wp.get_level(2, 'natural')])
print([node.path for node in wp.get_level(3, 'natural')])
print([node.path for node in wp.get_level(4, 'natural')])
```

['a', 'd']
['aa', 'ad', 'da', 'dd']
['aaa', 'aad', 'ada', 'add', 'daa', 'dad', 'dda', 'ddd']
['aaaa', 'aaad', 'aada', 'aadd', 'adaa', 'adad', 'adda', 'addd', 'daaa', 'daad', 'dada', 'dadd', 'ddaa', 'ddad', 'ddda', 'dddd']

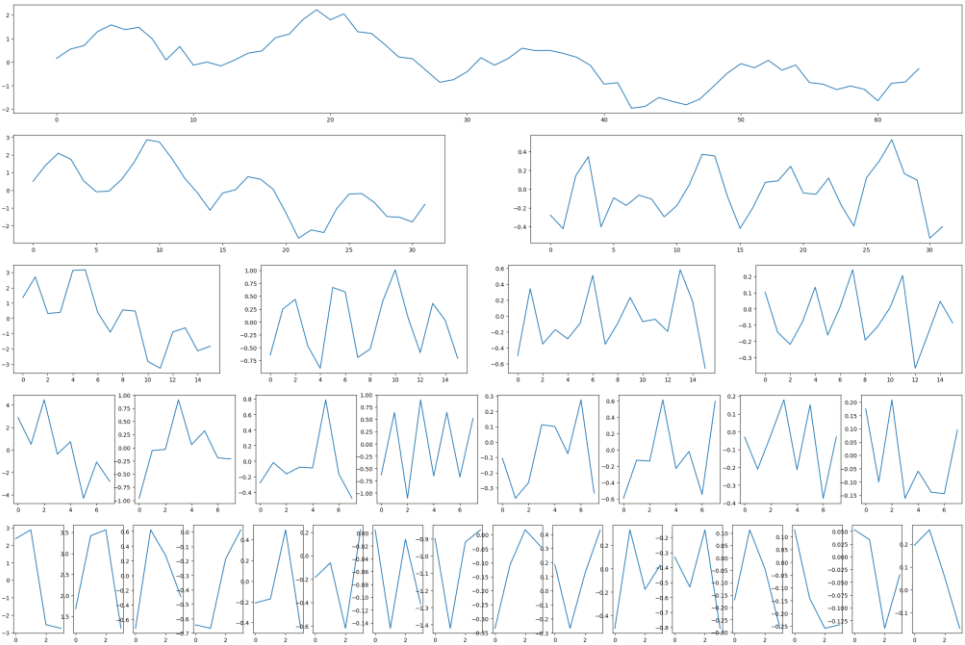
Для построения обозначим соответствующие узлы

```
#Первый уровень
a = wp['a'].data
d = wp['d'].data
#Второй уровень
aa = wp['aa'].data
ad = wp['ad'].data
da = wp['da'].data
dd = wp['dd'].data
#Третий уровень
aaa = wp['aaa'].data
aad = wp['aad'].data
…
ddd = wp['ddd'].data
```

```
#Четвёртый уровень
aaaa = wp['aaaa'].data
aaad = wp['aaad'].data
…
dddd = wp['dddd'].data
```

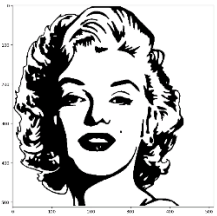
Выведем изображение уровней разложения

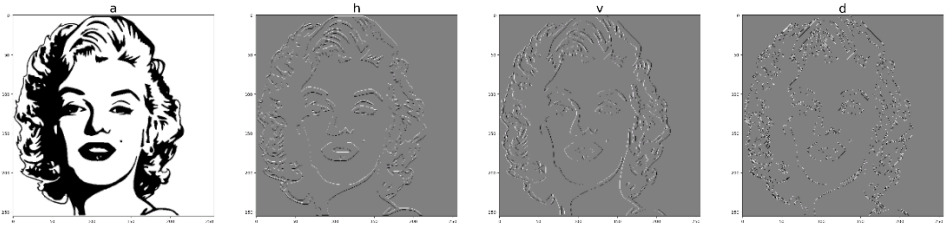
```
plt.figure(figsize=(30, 20))
#Оригинальный сигнал
plt.subplot(5,1,1)
plt.plot(x)
#Первый уровень
plt.subplot(5,2,3)
plt.plot(a)
plt.subplot(5,2,4)
plt.plot(d)
#Второй уровень
plt.subplot(5,4,9)
plt.plot(aa)
plt.subplot(5,4,10)
plt.plot(ad)
plt.subplot(5,4,11)
plt.plot(da)
plt.subplot(5,4,12)
plt.plot(dd)
#Третий уровень
plt.subplot(5,8,25)
plt.plot(aaa)
...
plt.plot(ddd)
#Четвёртый уровень
plt.subplot(5,16,65)
plt.plot(aaaa)
plt.subplot(5,16,66)
plt.plot(aaad)
…
plt.subplot(5,16,80)
plt.plot(dddd)
```

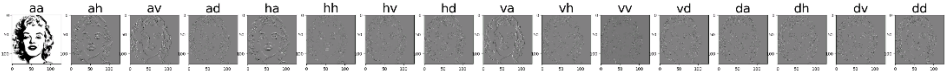


Изучая полученные изображения, можно видеть наилучшие уровни для каждой из задач — выделения особенностей, сжатия сигнала, устранения шумов.

Для обработки изображений используется функция Waveletpacket2D(). Для примера приведено разложение рисунка на 2 уровня. Можно заметить, что несколько «листьев» можно обнулить и наилучшим для сжатия будет использовать разноуровневое дерево.







Задания.

1. Разложите приведённые ниже сигналы интервале [0,4) с 1024 точками отсчёта, используя вейвлет-пакеты и 2-3 уровня разложения. Постройте изображения разложения на каждом уровне. Выберите наилучший базис сжатия, восстановите сигнал.
   a $\cos(2\cdot\pi\cdot t) + 4\cos(6\cdot\pi\cdot t) + r(t)$, где $r(t)$ - случайная компонента с равномерным распределением на отрезке [-0,5;0,5]
   b $4\cdot t - t^2 + r(t)$, где $r(t)$ – нормально распределённая случайная компонента с математическим ожиданием 0 и стандартным отклонением 0,5;

   Постройте графики исходного и восстановленного сигнала.
   Используйте вейвлет db1.
   Сравните с результатом параграфа 4
2. Проведите разложение выбранного Вами изображения с использованием двумерного вейвлет-пакета на 2 уровня. Постройте компоненты изображения. Выберите наилучшее дерево восстановления. Восстановите сигнал. Вейвлет выберите по своему желанию